

# Advanced Python

Day 12: Error Handling + Custom Exceptions

🕒 Duration: 2 Hours



# Real-Life Example: ATM Machine

-  **Wrong PIN** → Displays an error message
-  **No balance** → Shows "Insufficient balance"
-  **Card blocked** → "Access denied"

 **Concept:** Program crash nahi hota → Error handle hota hai!





# Without Error Handling

---



risky\_code.py

```
num = int(input("Number daalo: "))  
print(10 / num)
```

## ⚠ The Problems

- If user types 0 → **ZeroDivisionError**
- If user types "abc" → **ValueError**
- ✗ **Result:** Program crash ho jayega!



# Basic try-except

---

```
basic_try.py

try:
    num = int(input("Number daalo: "))
    result = 10 / num
    print(f"Result: {result}")

except ZeroDivisionError:
    print("❌ Zero se divide nahi kar sakte!")

except ValueError:
    print("❌ Sirf number daalo!")
```



# Using else & finally

```
advanced_try.py

try:
    num = int(input("Number daalo: "))
    result = 10 / num
except ZeroDivisionError:
    print("❌ Zero se divide nahi kar sakte!")
except ValueError:
    print("❌ Sirf number daalo!")
else:
    # Runs only if NO exception occurred
    print(f"✅ Result: {result}")
finally:
    # Always runs, crash ho ya na ho
    print("📋 Program khatam hua!")
```



# Output Examples

---

## Normal Input (5)

```
✓ Result: 2.0  
📋 Program khatam hua!
```

## Zero Input (0)

```
✗ Zero se divide nahi kar sakte!  
📋 Program khatam hua!
```



# Handling Multiple Exceptions

---



multi\_except.py

```
try:
    num = int(input("Number daalo: "))
    result = 10 / num

    # Catch multiple exceptions in a tuple
except (ZeroDivisionError, ValueError) as e:
    print(f"❌ Error aaya: {e}")

finally:
    print("📋 Done!")
```



# Common Built-in Errors

---



**ZeroDivisionError**

`10 / 0`



**ValueError**

`int("abc")`



**TypeError**

`"5" + 5`



**IndexError**

`my_list[10]`



**KeyError**

`my_dict["x"]`



**NameError**

`print(x)`



# Custom Exception Concept

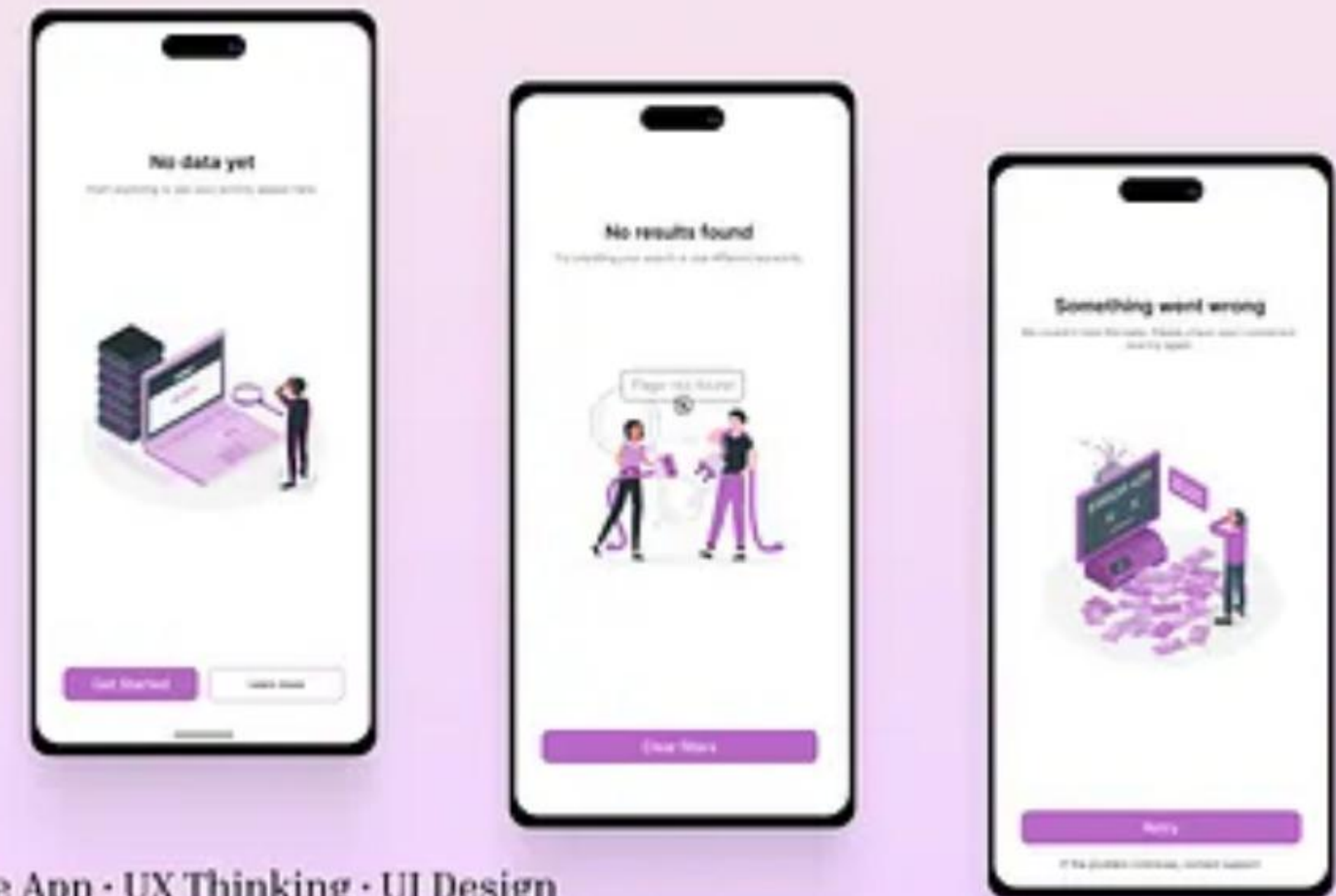
- ✓ Built-in errors exist for general Python issues.
- ? But what about **custom business logic** cases?

→ **Example:** A banking app where user requests more money than available.

We need an **InsufficientBalanceError**

## Empty & Error States UI

Clean UX for real-world scenarios



Mobile App • UX Thinking • UI Design



# Defining a Custom Exception

```
exceptions.py

# Inherit from the built-in Exception class
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount

    # Custom dynamic message
    message = f"❌ Balance kam hai! Available: ₹{balance}, Requested: ₹{amount}"

    # Initialize parent Exception
    super().__init__(message)
```



# BankAccount Class Logic

---

```
bank.py

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def withdraw(self, amount):
        # Validate before withdrawing
        if amount > self.balance:
            raise InsufficientBalanceError(self.balance, amount)

        self.balance -= amount
        print(f"✅ ₹{amount} nikaal liye! Bacha hua: ₹{self.balance}")
```



# Using the Custom Exception

```
main.py

acc = BankAccount("Saurav", 5000)

try:
    # This will succeed
    acc.withdraw(3000)

    # This will fail and RAISE our custom error
    acc.withdraw(8000)

except InsufficientBalanceError as e:
    print(e)

finally:
    print("📋 Transaction complete!")
```



# Transaction Output

---

✓ ₹3000 nikaal liye! Bacha hua: ₹2000

✗ Balance kam hai! Available: ₹2000, Requested: ₹8000

📋 Transaction complete!



# Another Example: Age Validation

---

```
age_validator.py

class InvalidAgeError(Exception):
    def __init__(self, age):
        message = f"❌ Invalid age: {age}! Age 0-150 ke beech honi chahiye."
        super().__init__(message)
```

Creating specific errors makes debugging incredibly easy.



# Function with Validation

---

```
functions.py

def set_age(age):
    # Check business rule
    if age < 0 or age > 150:
        # Throw custom error if rule is broken
        raise InvalidAgeError(age)

    print(f"✅ Age set: {age}")
```



# Using the Age Validation

---

```
app.py

try:
    # Valid input
    set_age(25)

    # Invalid input, triggers exception
    set_age(-5)

except InvalidAgeError as e:
    print(e)
```



# Validation Output

---

✓ Age set: 25

✗ Invalid age: -5! Age 0-150 ke beech honi chahiye.



# Structure Summary

---



**try**

Risky code runs here



**except**

Handle specific errors



**else**

Runs if NO error occurs



**finally**

Always runs at the end

**Custom Exception:**

```
class MyError(Exception): pass
```



# Practice: Grade System

grading.py (Part 1)

```
class InvalidMarksError(Exception):
    def __init__(self, marks):
        msg = f"❌ Invalid marks: {marks}! 0-100 only."
        super().__init__(msg)

def calculate_grade(marks):
    if marks < 0 or marks > 100:
        raise InvalidMarksError(marks)

    if marks >= 90:
        return "A+"
    elif marks >= 75:
        return "A"
    elif marks >= 60:
        return "B"
    elif marks >= 35:
        return "C"
    else:
        return "Fail"
```

grading.py (Part 2)

```
subjects = [85, 92, -10, 45, 110, 67]

for marks in subjects:
    try:
        grade = calculate_grade(marks)
        print(f"✅ Marks: {marks} → Grade: {grade}")
    except InvalidMarksError as e:
        print(e)

# Output Preview:
# ✅ Marks: 85 → Grade: A
# ✅ Marks: 92 → Grade: A+
# ❌ Invalid marks: -10! 0-100 only.
# ✅ Marks: 45 → Grade: C
# ❌ Invalid marks: 110! 0-100 only.
# ✅ Marks: 67 → Grade: B
```



# Practice Task

---

## Login System



### Step 1

Create InvalidUsernameError



### Step 2

Create InvalidPasswordError



### Step 3

Validate input & Raise exceptions accordingly



# Quick Revision

---

- ❓ **finally** kab run hota hai?
- ❓ Custom exception kis se **inherit** karta hai?
- ❓ ZeroDivisionError ka **output** kya hoga?



# Checklist

---

- ✓ **try-except** samjha
- ✓ **else-finally** samjha
- ✓ **Custom exceptions** banaye
- ✓ **Practice complete**



# Next Up

Day 13

Modules & Import System



# Image Sources

---



<https://media.istockphoto.com/id/169947046/photo/detailed-image-of-an-atm.jpg?s=612x612&w=0&k=20&c=bBDfNtsNI8NmvxOW31at5Qpwpf1uIUjESf5Ha6coitk=>

Source: [www.istockphoto.com](https://www.istockphoto.com)

---



Thumbnail

for

dribbble.com

<https://cdn.dribbble.com/userupload/46102267/file/2cea89764bfefdcbddaa2dbf71db227e.png?crop=0x0-5601x4201&format=webp&resize=400x300&vertical=center>

Source: [dribbble.com](https://dribbble.com)